TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

KHOA CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG

BÁO CÁO ĐỒ ÁN MÔN HỌC

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

ĐỀ TÀI: HỆ THỐNG QUẢN LÝ THƯ VIỆN & QUAN HỆ NGƯỜI MƯỢN SÁCH

(Quản lý quan hệ 1-N • Số lượng kho • 100% Cài đặt thủ công bằng Con trỏ C++)

Sinh viên thực hiện:	Tô Vĩnh Dũng
Môn học:	Cấu trúc Dữ liệu và Giải thuật
Mô hình dữ liệu:	Quan hệ Sách — Phiếu mượn Sinh viên
Ngôn ngữ:	C++ (Chuẩn C++11 • Không dùng STL)

HÀ NỘI — 2026

1. GIỚI THIỆU TỔNG QUAN VÀ MÔ HÌNH BÀI TOÁN

Trong thực tế vận hành thư viện trường đại học, mỗi đầu sách luôn có **tổng số lượng (inventory)** nhất định và có thể cho **nhiều sinh viên mượn đồng thời** cho đến khi số lượng còn lại trong kho bằng 0. Hệ thống cần ghi nhận chi tiết: *Ai đang mượn? (Họ tên, Mã số sinh viên - MSSV), mượn cuốn sách nào, loại sách gì và vào thời điểm nào?*

> **Mô hình quan hệ 1 – N (Một – Nhiều):**
> Một **Đầu Sách** (Book) $\rightarrow$ Nhiều **Phiếu Mượn** (Borrower Records). Khi sinh viên mượn sách, hệ thống giảm `soLuongConLai--`. Khi `soLuongConLai == 0`, cuốn sách chính thức hết và các sinh viên đến sau sẽ được xếp vào **Hàng đợi đặt trước (Queue FIFO)**.

2. CẤU TRÚC DỮ LIỆU TỰ THIẾT KẾ

2.1. Cấu trúc Đầu Sách (`struct Book`)

```
struct Book {
    string maSach;          // Mã định danh duy nhất (ví dụ: "CS101")
    string tenSach;         // Tên cuốn sách
    string loaiSach;        // Thể loại: "Giáo trình", "Kỹ thuật", "Tham khảo"...
    int tongSoLuong;        // Tổng số bản sách thư viện sở hữu
    int soLuongConLai;      // Số lượng sách hiện còn trong kho (cho mượn nếu > 0
    int luotMuon;           // Tổng số lượt sách đã được mượn
};
```

2.2. Cấu trúc Phiếu Mượn của Sinh viên (`struct PhieuMuon`)

```
struct PhieuMuon {
    string maPhieu;         // Mã phiếu: "PM001", "PM002"...
    string tenSinhVien;     // Họ và tên sinh viên mượn sách
    string maSinhVien;      // Mã số sinh viên (MSSV)
    string maSach;          // Mã sách mượn
    string tenSach;         // Tên cuốn sách mượn
    string loaiSach;        // Thể loại sách
    string ngayMuon;        // Ngày lập phiếu mượn (ví dụ: "16/10/2026")
};

struct NodePhieuMuon {
    PhieuMuon data;
    NodePhieuMuon* next;    // Con trỏ liên kết danh sách phiếu mượn
};
```

2.3. Ngăn xếp Hoàn tác (`struct Stack` LIFO)

```
enum ActionType { ACTION_MUON_SACH, ACTION_XOA_SACH };

struct UndoAction {
    ActionType loaiHanhDong;
    Book sach;              // Lưu bản sao sách
    PhieuMuon phieu;        // Lưu thông tin phiếu mượn (để xóa khi Undo mượn)
};

struct StackNode {
    UndoAction data;
    StackNode* next;
};

struct Stack {
    StackNode* top;         // Con trỏ đỉnh ngăn xếp LIFO
    int count;
};
```

2.4. Hàng đợi Đặt trước (`struct Queue` FIFO)

```
struct PhieuDatCho {
    string tenSinhVien;     // Họ tên sinh viên chờ mượn
    string maSinhVien;      // MSSV
    string maSachCanMuon;   // Mã sách cần mượn khi có sách hoàn kho
};

struct QueueNode {
    PhieuDatCho data;
    QueueNode* next;
};

struct Queue {
    QueueNode* front;       // Đầu hàng đợi (lấy ra phục vụ)
    QueueNode* rear;        // Cuối hàng đợi (thêm người vào)
    int count;
};
```

3. CHI TIẾT CÁC THUẬT TOÁN SỬ DỤNG VÀ NGUYÊN LÝ HOẠT ĐỘNG

3.1. Thuật toán Mượn sách (Quy trình kiểm tra số lượng và tạo phiếu)

1. Người dùng nhập mã sách cần mượn. Hệ thống tìm kiếm nhanh bằng **Binary Search $O(\log n)$**.
2. Kiểm tra điều kiện:
   - **Nếu `soLuongConLai > 0`:** Cho phép mượn. Yêu cầu nhập Họ tên sinh viên và MSSV. Hệ thống giảm `soLuongConLai--`, tăng `luotMuon++`, tự sinh mã phiếu `PMxxx` và chèn một nút `NodePhieuMuon` mới vào cuối danh sách liên kết phiếu mượn. Đẩy thao tác vào đỉnh **Stack Undo**.
   - **Nếu `soLuongConLai == 0`:** Sách đã được mượn hết. Hệ thống từ chối cho mượn và đưa sinh viên vào cuối **Hàng đợi đặt trước (Queue FIFO)**.

3.2. Thuật toán Nhận trả sách & Ưu tiên Hàng đợi

1. Nhập MSSV và Mã sách trả. Duyệt danh sách liên kết phiếu mượn, hủy nút tương ứng.
2. Tăng số lượng sách trong kho: `soLuongConLai++`.
3. Kiểm tra Hàng đợi đặt trước: Nếu `queueDatCho` có sinh viên đang đợi đúng mã sách này, hệ thống phát tín hiệu ưu tiên giải quyết ngay cho sinh viên đứng đầu hàng đợi (Dequeue).

3.3. Thuật toán Hoàn tác (Undo) theo mô hình quan hệ

- **Hoàn tác thao tác Mượn sách:** Rút hành động từ đỉnh Stack ra (`pop`). Hệ thống tự động xóa phiếu mượn của sinh viên đó khỏi danh sách liên kết phiếu mượn, đồng thời hoàn trả lại `soLuongConLai++` và giảm `luotMuon--`.
- **Hoàn tác thao tác Xóa sách:** Khôi phục lại toàn bộ đầu sách vào kho.

3.4. Thuật toán Sắp xếp QuickSort & Tìm kiếm Nhị phân

Do danh sách liên kết đơn không hỗ trợ truy cập ngẫu nhiên ($O(1)$ random access), chương trình chuyển các phần tử sang một mảng động `Book* arr = new Book[n]`, chạy thuật toán **QuickSort (phân hoạch Lomuto)** để sắp xếp tăng dần theo mã sách, sau đó áp dụng **Binary Search** với độ phức tạp $O(\log n)$ để tìm kiếm mã sách tức thì.

4. BẢNG TỔNG HỢP ĐỘ PHỨC TẠP THUẬT TOÁN

THAO TÁC NGHIỆP VỤ	CẤU TRÚC DỮ LIỆU SỬ DỤNG	ĐỘ PHỨC TẠP THỜI GIAN (TIME)	ĐỘ PHỨC TẠP KHÔNG GIAN (SPACE)
Thêm đầu sách mới	Singly Linked List (Sách)	$O(n)$	$O(1)$
Lập phiếu mượn sinh viên	Singly Linked List (Phiếu mượn)	$O(n)$	$O(1)$
Trả sách & Hủy phiếu mượn	Singly Linked List (Phiếu mượn)	$O(n)$	$O(1)$
Lưu Undo / Hoàn tác	Stack (LIFO con trỏ top)	$O(1)$	$O(1)$
Đặt chỗ khi hết sách	Queue (FIFO con trỏ rear)	$O(1)$	$O(1)$
Phục vụ hàng đợi (Dequeue)	Queue (FIFO con trỏ front)	$O(1)$	$O(1)$
Sắp xếp danh mục sách	Mảng động + QuickSort	Trung bình: $O(n \log n)$	$O(\log n)$
Tìm kiếm mã sách	Mảng động + Binary Search	$O(\log n)$	$O(1)$
Xuất Web HTML tĩnh	File Stream + Linked Lists	$O(n + m)$ (n: sách, m: phiếu)	$O(1)$

5. HỎI ĐÁP PHỤC VỤ BẢO VỆ ĐỒ ÁN TRƯỚC HỘI ĐỒNG

Câu hỏi 1: Em thiết kế quan hệ giữa Sách và Người mượn trong bài toán này như thế nào?

Trả lời: Em thiết kế theo mô hình quan hệ **1 – N (Một – Nhiều)**. Mỗi đầu sách trong thư viện có tổng số lượng `tongSoLuong` và số lượng hiện còn `soLuongConLai`. Khi sinh viên mượn sách, một bản ghi `PhieuMuon` được tạo ra chứa đầy đủ Họ tên, MSSV, Mã sách và Loại sách. Một cuốn sách có thể được nhiều sinh viên mượn cùng lúc cho đến khi `soLuongConLai == 0`.

Câu hỏi 2: Tại sao em dùng 2 Danh sách liên kết đơn riêng biệt thay vì lồng nhau?

Trả lời: Em tách thành 2 Danh sách liên kết đơn độc lập: một DSLK quản lý danh mục sách (Kho sách) và một DSLK quản lý danh sách phiếu mượn (Nhật ký mượn). Cách thiết kế này tuân theo nguyên lý phân tách trách nhiệm (Separation of Concerns), giúp việc quản lý kho sách, sắp xếp bằng QuickSort và tìm kiếm nhị phân được độc lập, đồng thời việc tra cứu sinh viên đang mượn sách không làm cồng kềnh cấu trúc cuốn sách.

Câu hỏi 3: Khi số lượng sách còn lại bằng 0 thì hệ thống xử lý ra sao?

Trả lời: Khi `soLuongConLai == 0`, hệ thống từ chối cho mượn và tự động kích hoạt chức năng đưa sinh viên vào Hàng đợi đặt trước (Queue FIFO). Khi có bất kỳ sinh viên nào đem trả cuốn sách đó lại kho, hệ thống kiểm tra ngay đỉnh Hàng đợi để thông báo ưu tiên mượn cho người đăng ký sớm nhất.

6. KẾT LUẬN

Hệ thống đã giải quyết trọn vẹn bài toán nghiệp vụ thư viện thực tế với mô hình quan hệ Sách – Sinh viên mượn sách, áp dụng thành công các cấu trúc dữ liệu nền tảng (Singly Linked List, Stack LIFO, Queue FIFO, QuickSort, Binary Search) được lập trình 100% bằng con trỏ thuần trong C++.